

Servlets



BÀI 08: LẬP TRÌNH SERVLET

- ❑ Khái niệm Servlet
- ❑ Vòng đời Servlet
- ❑ Cấu hình Servlet
- ❑ Servlet Url Pattern
- ❑ Bài tập



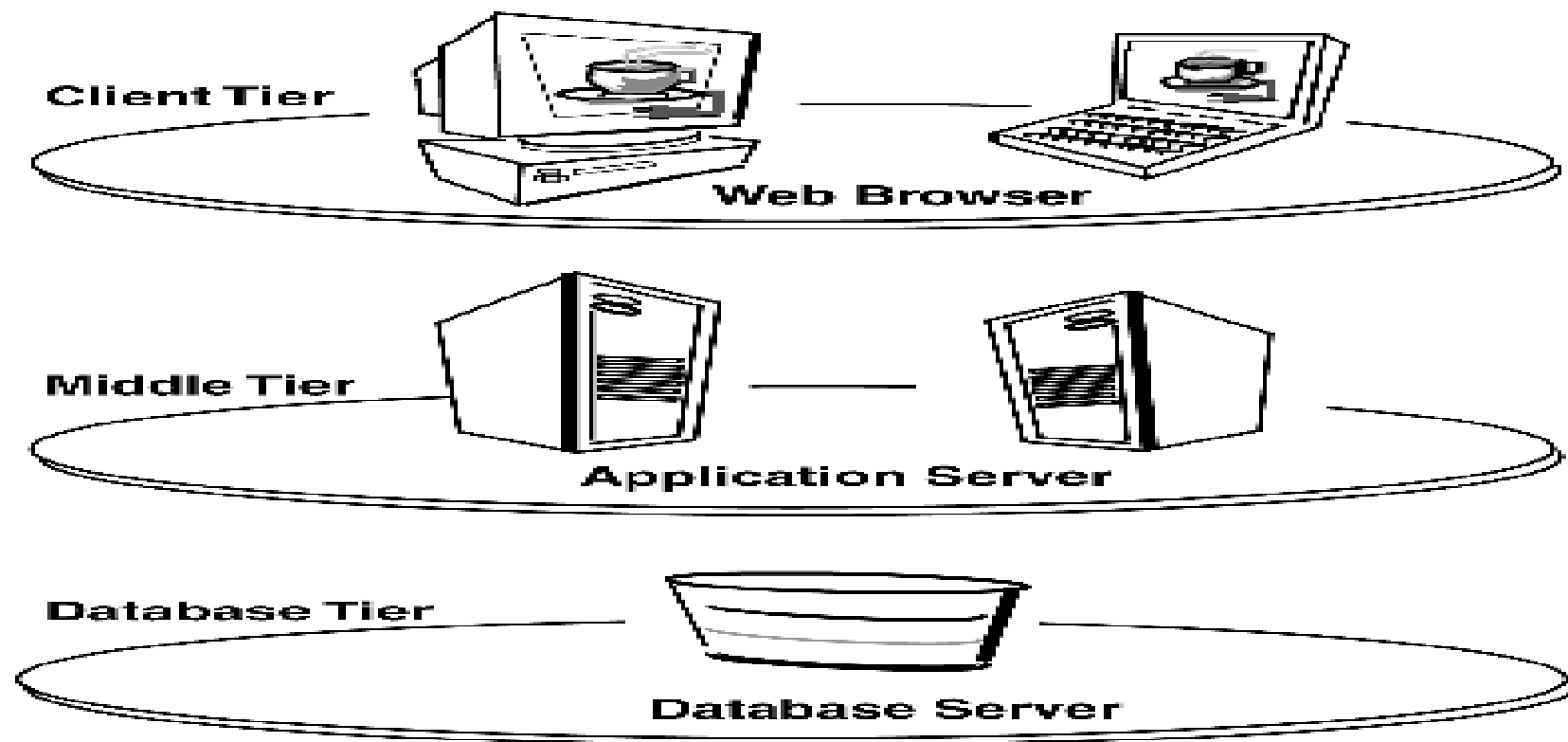
SERVLETS

Java Servlets là các chương trình chạy trên một Web server hoặc một Application server và thực hiện như là một tầng trung gian giữa một Yêu cầu từ một trình duyệt web hoặc HTTP client với các Database hoặc các ứng dụng trên HTTP server.

- ✓ Sử dụng Servlets, ta có thể thu thập Input từ người sử dụng thông qua các form trên trang web
- ✓ Hiển thị các bản ghi (record) từ một Database hoặc từ nguồn khác
- ✓ Tạo các trang web động.

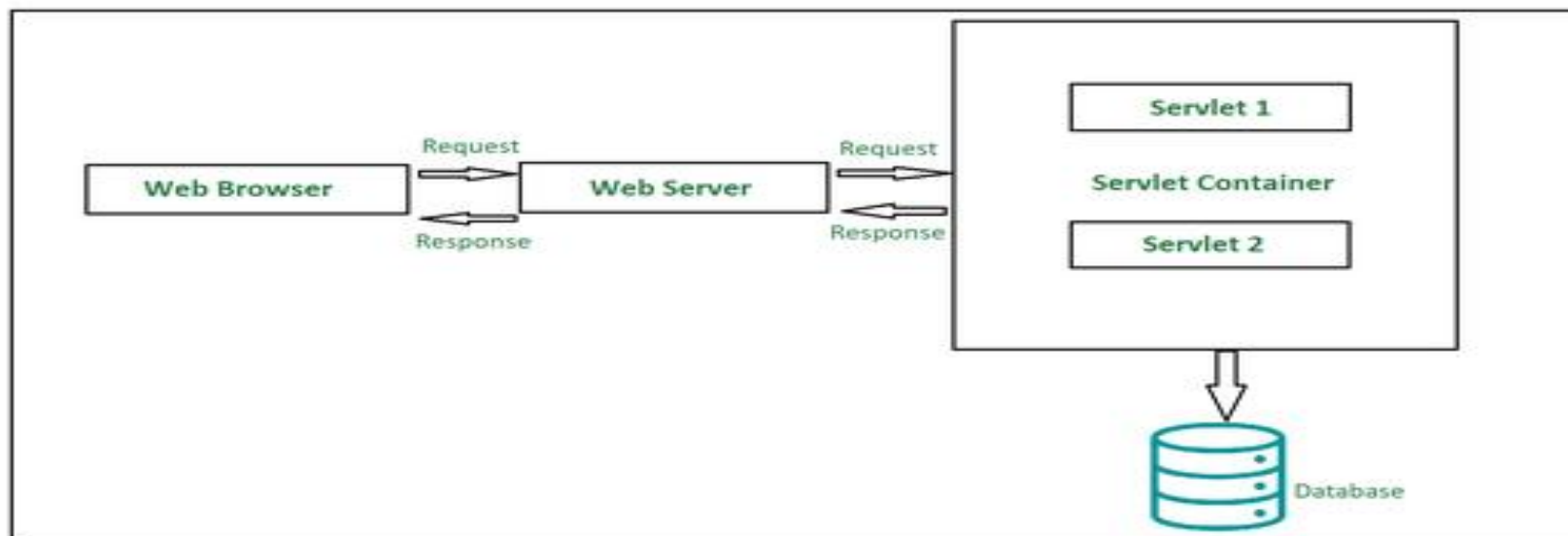


SERVLETS

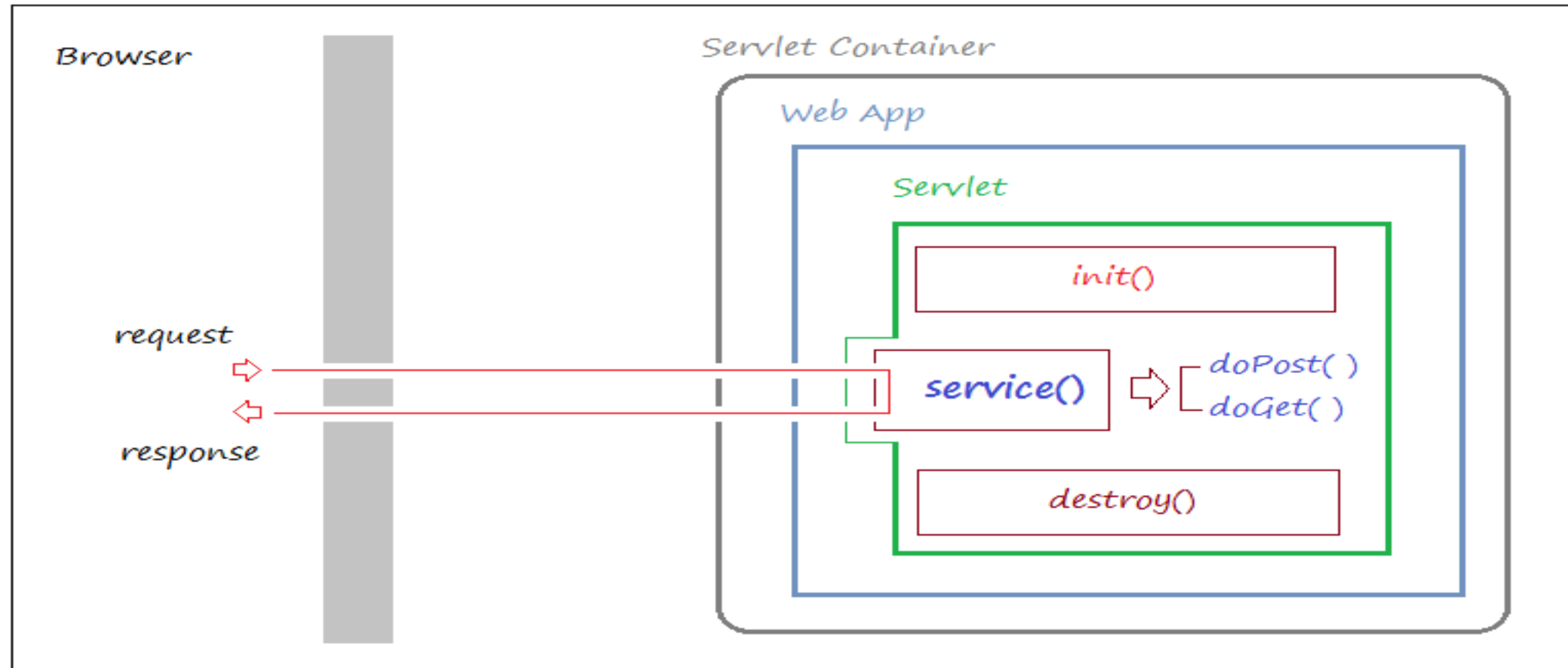




SERVLETS



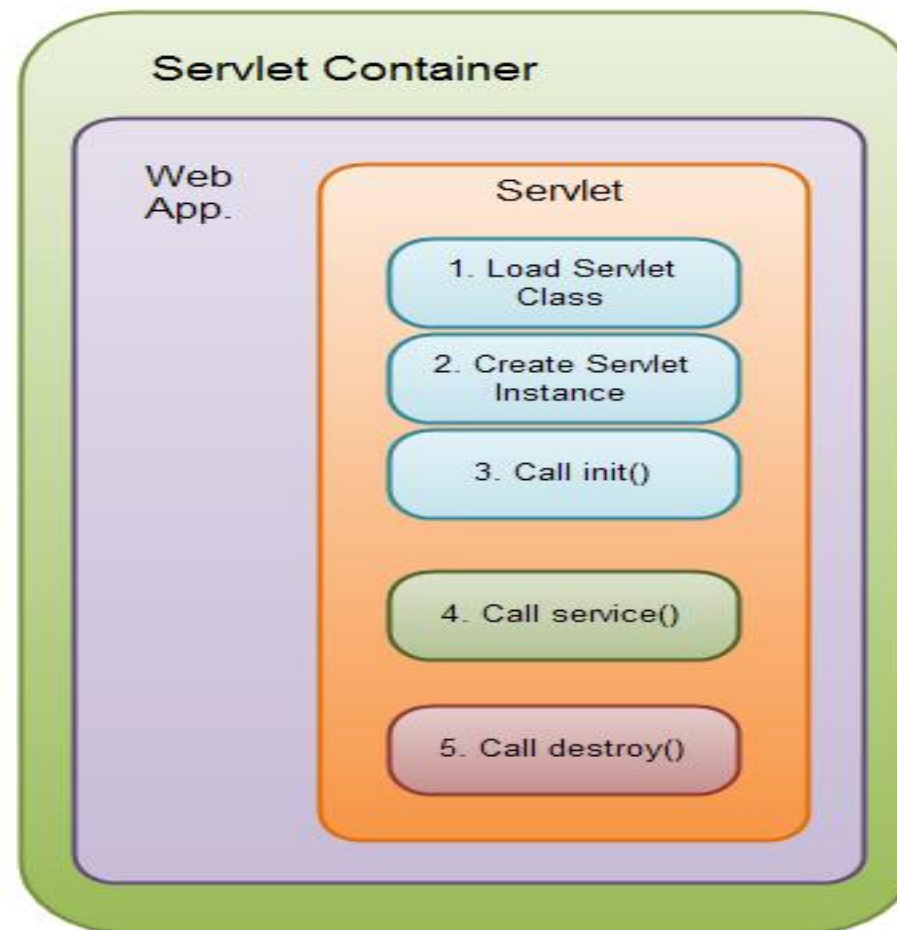
Servlets





VÒNG ĐỜI CỦA SERVLET

1. Tải lớp Servlet vào bộ nhớ.
2. Tạo đối tượng Servlet.
3. Gọi phương thức `init()` của Servlet.
4. Gọi phương thức `service()` của Servlet.
5. Gọi phương thức `destroy()` của Servlet.





CÀI ĐẶT MÔI TRƯỜNG

❑ Cài đặt môi trường cần thiết

- ✓ **Java Development Kit (JDK):** Đảm bảo JDK đã được cài đặt và cấu hình.
- ✓ **Apache Tomcat:** Cài đặt Apache Tomcat (hoặc bất kỳ máy chủ web nào hỗ trợ Servlet).
- ✓ **Eclipse IDE:** Sử dụng phiên bản Eclipse hỗ trợ phát triển Java EE (Enterprise Edition).



CÀI ĐẶT MÔI TRƯỜNG

❑ Cấu hình máy chủ Apache Tomcat trên Eclipse

- ✓ Mở Eclipse, chọn **Window > Preferences**.
- ✓ Vào **Server > Runtime Environments**.
- ✓ Nhấn **Add**, chọn **Apache Tomcat** và phiên bản phù hợp (ví dụ: Tomcat 9).
- ✓ Trở đến thư mục cài đặt của Tomcat trên máy và nhấn **Finish**.



CÀI ĐẶT MÔI TRƯỜNG

- ❑ Xây dựng chương trình Servlet đầu tiên





CÀI ĐẶT MÔI TRƯỜNG

- ❑ Cấu hình Eclipse để chạy ứng dụng trên tomcat
- ✓ *Chú ý:*
- ✓ */ServletTutorial được gọi là **Context-Path**, mỗi website đều có **Context-Path**, ta cũng có thể cấu hình cho nó một giá trị khác, hoặc để rỗng. Trong trường hợp rỗng có thể truy cập vào web theo cách:*
 - *http://localhost:8080*
 - *http://localhost:8080/index.html*
- ✓ *Khi chạy với Tomcat mặc định nó lấy tên Project làm **Context-Path**.*

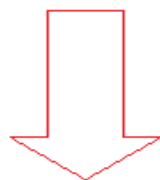


Cấu hình Eclipse để chạy ứng dụng trên tomcat

Client

Browser:

http://localhost:8080/ServletTutorial/

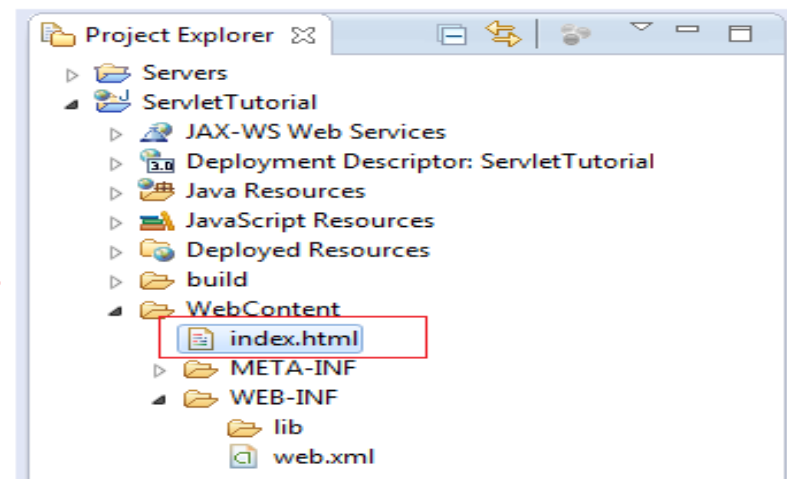
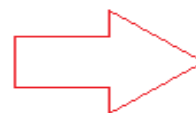


index.html

Server

web.xml

```
<welcome-file-list>
  <welcome-file>index.html</welcome-file>
  <welcome-file>index.htm</welcome-file>
  <welcome-file>index.jsp</welcome-file>
  <welcome-file>default.html</welcome-file>
  <welcome-file>default.htm</welcome-file>
  <welcome-file>default.jsp</welcome-file>
</welcome-file-list>
```

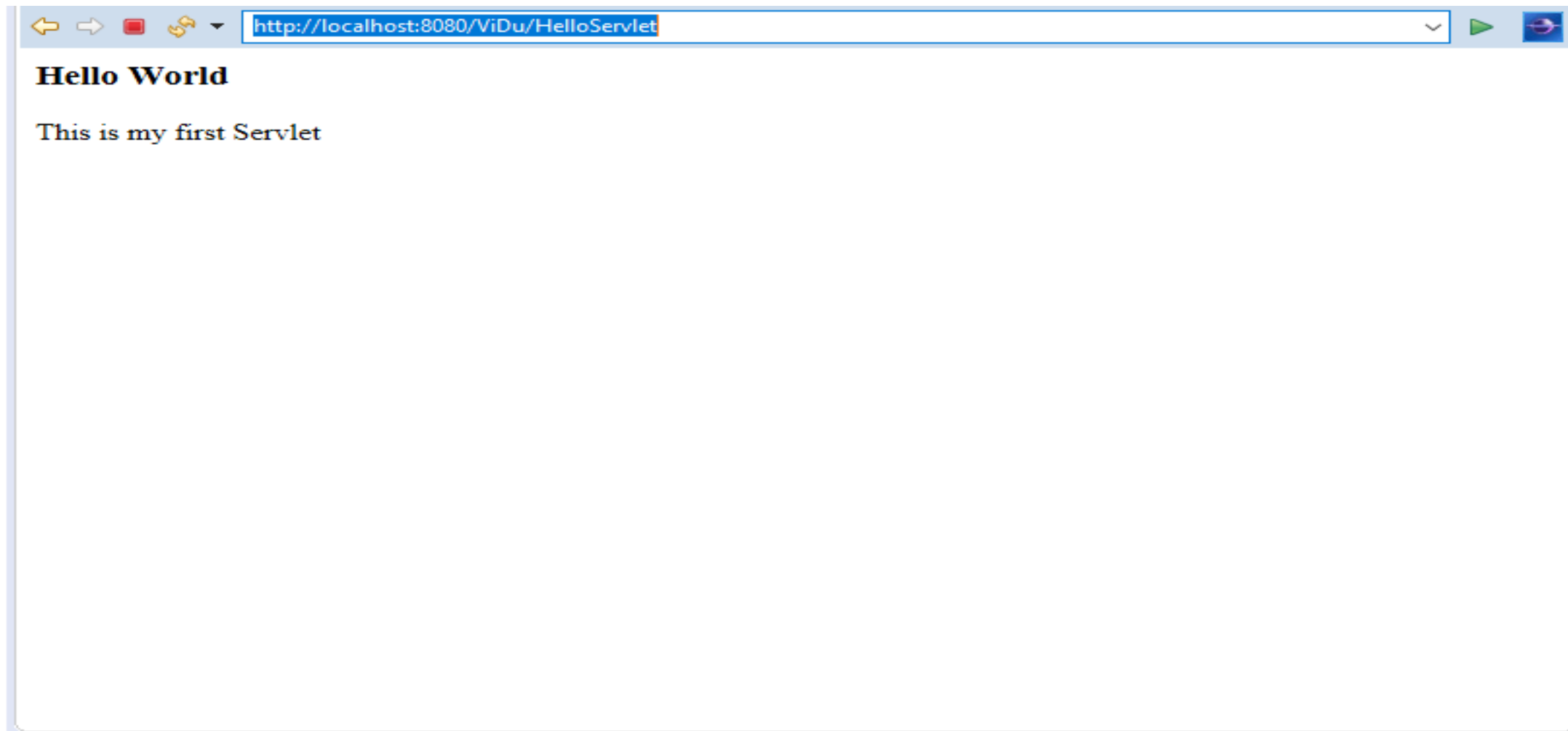


Chương trình hello

```
package abc.cdf;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/HelloServlet")
public class HelloServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    public HelloServlet() {
        super();
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        out.println("<html>");
        out.println("<head><title>Hello Servlet</title></head>");
        out.println("<body>");
        out.println("<h3>Hello World</h3>");
        out.println("This is my first Servlet");
        out.println("</body>");
        out.println("<html>");
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```



Chương trình hello





CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Mục đích của cấu hình Servlet

✓ Ánh xạ URL đến Servlet

Cấu hình giúp ánh xạ các đường dẫn URL cụ thể trong ứng dụng tới các lớp Servlet xử lý logic tương ứng. Ví dụ, /login có thể ánh xạ tới một Servlet xử lý việc đăng nhập.

✓ Quản lý tham số khởi tạo

Cấu hình cho phép truyền các tham số tĩnh (chẳng hạn thông tin cấu hình như URL cơ sở dữ liệu) tới các Servlet mà không cần sửa mã nguồn.

✓ Định nghĩa hành vi cụ thể cho từng Servlet

Mỗi Servlet có thể được cấu hình để phục vụ các mục đích khác nhau, chẳng hạn xử lý GET, POST hoặc các hành động tùy chỉnh khác.

✓ Hỗ trợ bảo mật và quản lý quyền truy cập

Các cấu hình có thể tích hợp với các chính sách bảo mật, hạn chế quyền truy cập vào một Servlet dựa trên vai trò người dùng.

✓ Trong Java Servlet, cấu hình có thể được thực hiện theo hai cách:

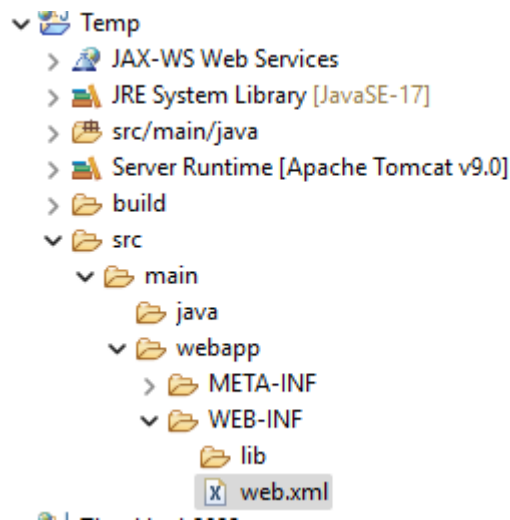
- Sử dụng web.xml
- Sử dụng Annotation (@WebServlet)



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Cấu hình Servlet bằng web.xml

Sử dụng tệp web.xml nằm trong thư mục WEB-INF của ứng dụng.



```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
  <!-- Khai báo Servlet -->
  <servlet>
    <servlet-name>ExampleServlet</servlet-name>
    <servlet-class>com.example.ExampleServlet</servlet-class>
    <init-param>
      <param-name>dbURL</param-name>
      <param-value>jdbc:mysql://localhost:3306/testdb</param-value>
    </init-param>
  </servlet>

  <!-- Ánh xạ URL -->
  <servlet-mapping>
    <servlet-name>ExampleServlet</servlet-name>
    <url-pattern>/example</url-pattern>
  </servlet-mapping>
</web-app>
```



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Cấu hình Servlet bằng web.xml

✓ Ý nghĩa của các thẻ trong web.xml

- `<servlet-name>`: Tên định danh của Servlet (có thể tự đặt).
- `<servlet-class>`: Đường dẫn đầy đủ tới lớp Servlet.
- `<init-param>`: Khai báo tham số khởi tạo cho Servlet.
- `<servlet-mapping>`: Gắn Servlet với một hoặc nhiều URL (/example).



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Cấu hình Servlet bằng Annotation

- ✓ Sử dụng Annotation @WebServlet trong mã nguồn Servlet:

```
import javax.servlet.annotation.WebInitParam;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet(
    name = "ExampleServlet",
    urlPatterns = {"/example"},
    initParams = {
        @WebInitParam(name = "dbURL", value = "jdbc:mysql://localhost:3306/testdb")
    }
)
public class ExampleServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.getWriter().write("Hello from ExampleServlet!");
    }
}
```



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Cấu hình Servlet bằng Annotation

- ✓ Sử dụng Annotation @WebServlet trong mã nguồn Servlet:

```
import javax.servlet.annotation.WebInitParam;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet(
    name = "ExampleServlet",
    urlPatterns = {"/example"},
    initParams = {
        @WebInitParam(name = "dbURL", value = "jdbc:mysql://localhost:3306/testdb")
    }
)
public class ExampleServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.getWriter().write("Hello from ExampleServlet!");
    }
}
```



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ Cấu hình Servlet bằng Annotation

✓ Ý nghĩa của các Annotation

- `@WebServlet`: Annotation dùng để định nghĩa Servlet.
- `@WebInitParam`: Annotation dùng cho tham số khởi tạo
- `name`: Tên Servlet (tùy chọn).
- `urlPatterns`: Danh sách URL ánh xạ tới Servlet.
- `initParams`: Khai báo tham số khởi tạo.



CẤU HÌNH SERVLET SỬ DỤNG WEB.XML VÀ ANNOTATION

❑ So sánh web.xml và Annotation

Đặc điểm	web.xml	Annotation
Độ phức tạp	Phức tạp hơn, yêu cầu chỉnh sửa XML.	Dễ sử dụng, cấu hình trực tiếp.
Quản lý cấu hình	Tốt hơn cho ứng dụng lớn.	Phù hợp cho ứng dụng nhỏ.
Mức độ linh hoạt	Cao (hỗ trợ ánh xạ nhiều URL).	Ít linh hoạt hơn.
Tính hiện đại	Cũ hơn, cần trong các ứng dụng cũ.	Mới, áp dụng từ Servlet 3.0 trở đi.



THAM SỐ KHỞI TẠO SERVLET

- ❑ Tham số khởi tạo giúp quản lý cấu hình ứng dụng một cách tập trung và linh hoạt, tách biệt mã nguồn và thông tin cấu hình.
- ❑ Cung cấp dữ liệu cần thiết ngay khi Servlet khởi tạo, giảm phụ thuộc vào mã nguồn.
- ❑ Các phương thức thông qua đối tượng ServletConfig config
 - ✓ **getInitParameter(String name)**: Trả về giá trị của tham số khởi tạo theo tên.
 - ✓ **getInitParameterNames()**: Trả về danh sách các tên tham số khởi tạo.



THAM SỔ KHỞI TẠO SERVLET

ServletConfig

```
import java.io.IOException;

import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class Init_ExampleServlet extends HttpServlet {
    private String dbURL;
    private String dbUser;

    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        dbURL = config.getInitParameter("dbURL");
        dbUser = config.getInitParameter("dbUser");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        resp.getWriter().write("Database URL: " + dbURL + ", User: " + dbUser);
    }
}
```



SERVLET URL PATTERN

Servlet *url-pattern* = */spath/**

contextPath

servletPath

pathInfo

queryString

http://localhost:8080/ServletTutorial/spath/abc/mnp?p1=1

requestURL

requestURI

Khi người dùng nhập vào một đường dẫn trên trình duyệt, nó sẽ được gửi tới WebContainer. WebContainer cần phải quyết định xem Servlet nào sẽ phục vụ yêu cầu này từ phía người dùng.



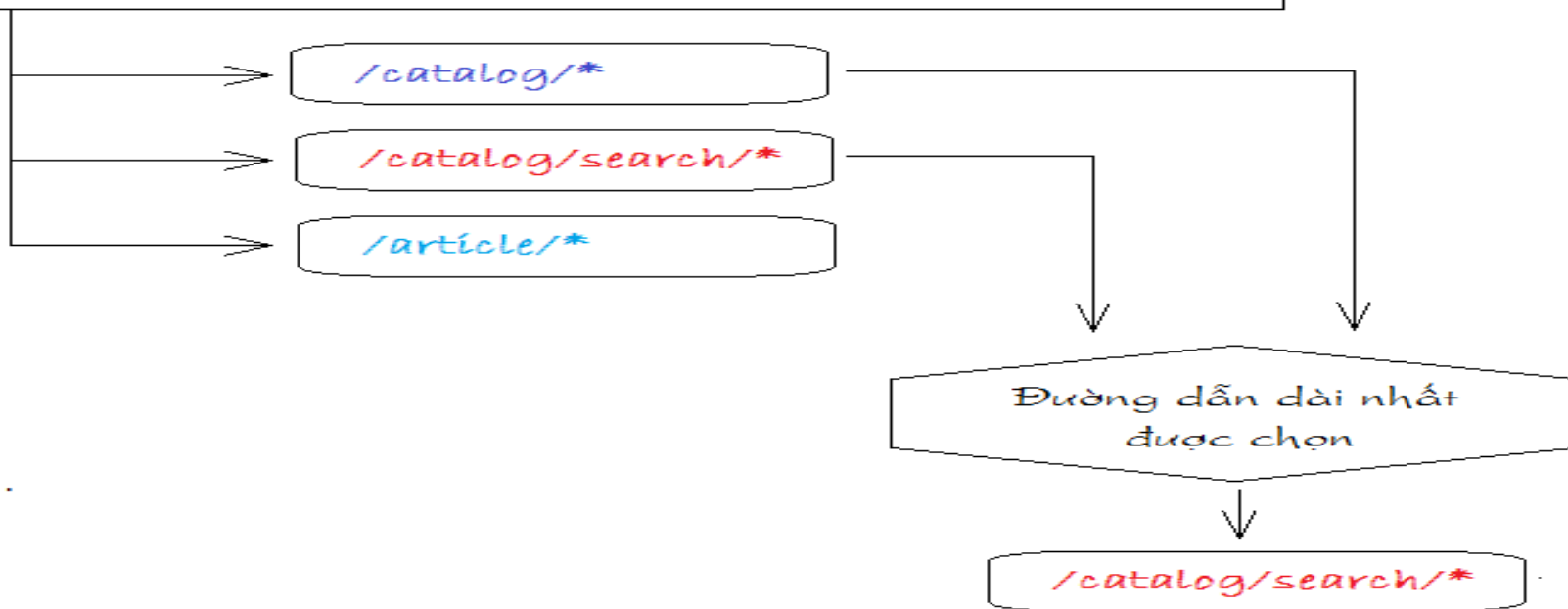
SERVLET URL PATTERN

Servlet1: `/catalog/*`

Servlet2: `/catalog/search/*`

Servlet3: `/article/*`

`http://example.com/contextPath/catalog/search/product?name=abc`





SERVLET URL PATTERN

AsteriskServlet

<http://localhost:8080/ViDu/any/path1/a.html>

<http://localhost:8080/ViDu/any/something>

<http://localhost:8080/ViDu/any>



SERVLET URL PATTERN

- ❑ Servlet mặc định:
- ✓ Servlet với url-pattern = /
- ✓ Là một **servlet** mặc định, **servlet** này sẽ được sử dụng để xử lý các yêu cầu (request) mà có đường dẫn không khớp với bất kỳ một ***url-pattern*** nào của các **Servlet** khác được khai báo trong ứng dụng



LẤY CÁC THÔNG TIN CƠ BẢN CỦA SERVLET

<http://localhost:8080/ViDu/other/exampleInfo?text1=Hello&text2=World>

```
public class InfoServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {  
        // Lấy thông tin từ HttpServletRequest  
        String clientIP = req.getRemoteAddr();  
        String uri = req.getRequestURI();  
        String method = req.getMethod();  
  
        // Lấy thông tin từ ServletConfig  
        ServletConfig config = getServletConfig();  
        String servletName = config.getServletName();  
  
        // Lấy thông tin từ ServletContext  
        ServletContext context = getServletContext();  
        String appName = context.getInitParameter("appName");  
    }  
}
```



LẬP TRÌNH SERVLET

☐ Bài tập thực hành

Bài 1: Hãy tạo một Servlet để hiển thị thông tin Welcome với URL pattern được cấu hình trong web.xml. Servlet này cần đọc một tham số khởi tạo từ web.xml và hiển thị thông tin đó trong trình duyệt.

Yêu cầu:

1. Tạo một dự án Java Dynamic Web Project trên Eclipse.
2. Cấu hình web.xml để ánh xạ Servlet với URL pattern /welcome.
3. Thêm tham số khởi tạo vào web.xml, ví dụ: welcomeMessage = "Welcome to Servlet Basics!".
4. Sử dụng phương thức doGet() để đọc tham số khởi tạo và hiển thị thông tin trong trình duyệt.
5. Triển khai và chạy ứng dụng trên máy chủ Tomcat.



LẬP TRÌNH SERVLET

❑ Bài tập thực hành

Bài 1: Tạo một Servlet sử dụng annotation thay cho cấu hình web.xml. Servlet này sẽ xử lý yêu cầu POST từ một form HTML, lấy thông tin nhập vào từ người dùng, và phản hồi thông tin đó.

Yêu cầu:

1. Tạo một Servlet sử dụng annotation với URL pattern /submit.
2. Sử dụng phương thức doPost() để nhận dữ liệu từ form HTML.
3. Tạo một form HTML đơn giản để gửi dữ liệu (tên và email) đến Servlet.
4. Servlet trả về phản hồi hiển thị tên và email của người dùng.
5. Triển khai và kiểm tra trên máy chủ Tomcat.



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM